RelocateHub

Comprehensive Project Report

Full-Stack Relocation Platform — Technical & Functional Overview

Project Name	RelocateHub
Version	2.0 (version2)
Report Date	May 30, 2026
License	MIT License (Open Source)
Repository	https://github.com/mpriyanshu22/RelocateHelp.git
Category	Full-Stack Web Application
Status	Active Development

1. Executive Summary

RelocateHub is a full-stack, production-ready web application designed to simplify the relocation process for individuals and families. The platform acts as a single, consolidated destination for anyone moving to a new city, offering structured city guides, searchable service listings, a user review system, and a secure admin control panel.

Built on a modern JavaScript ecosystem — React.js on the frontend and Node.js/Express on the backend with MongoDB as the data store — RelocateHub follows industry best practices for authentication (JWT via HTTP-only cookies), secure password handling (bcrypt), and RESTful API design.

Key strengths of the platform include:

- A clean, component-driven frontend built with React and Tailwind CSS
- Secure, stateless authentication using JSON Web Tokens stored in cookies
- Scalable MongoDB data models for cities, listings, users, and reviews
- A dedicated Admin Control Panel for platform governance
- Advanced search and filter capabilities for service discovery

2. Technology Stack

RelocateHub leverages a modern MERN-adjacent stack, carefully chosen for scalability, developer productivity, and community support.

2.1 Frontend

React.js (Vite) Component-based UI framework with lightning-fast HMR via Vite	**Tailwind CSS** Utility-first CSS framework for responsive, maintainable styling
React Router Client-side routing for a seamless single-page application experience	**Lucide React** Consistent, modern icon library with tree-shakable SVG icons
Axios Promise-based HTTP client for all API communication with interceptors	

2.2 Backend

Node.js Event-driven JavaScript runtime powering the server	**Express.js** Minimal, unopinionated web framework for REST API development
MongoDB NoSQL document database for flexible schema design at scale	**Mongoose** Elegant ODM providing schema validation, middleware, and query helpers
JWT (Cookies) Stateless authentication tokens stored securely in HTTP-only cookies	**Bcrypt** Industry-standard adaptive hashing algorithm for password security

2.3 Architecture Overview

The application follows a classic client-server architecture with clear separation of concerns:

Layer	Technology	Responsibility
Presentation	React.js + Tailwind CSS	UI rendering, routing, state management, responsive design
API Client	Axios	HTTP requests, response interceptors, cookie handling
API Server	Node.js + Express.js	Route handling, business logic, middleware processing
Auth Layer	JWT + Bcrypt	Token generation, validation, password hashing
Data Layer	MongoDB + Mongoose	Data persistence, schema validation, query optimisation

3. Key Features

RelocateHub delivers six core feature modules, each targeting a distinct aspect of the relocation user journey.

Feature	Description	User Role
Dynamic City Guides	Explore destination cities including cost of living breakdowns, key neighborhoods, and essential local information to help users evaluate relocation targets.	All Users
Service Listings Filter & Search	Search and filter across housing, schools, and healthcare categories. Supports advanced filtering by category and rating, plus real-time popularity sorting.	All Users
User Authentication	Secure registration and login with JWT tokens stored in HTTP-only cookies for persistent, XSS-resistant sessions. Passwords are hashed with bcrypt.	Registered Users
User Dashboard	Personalized dashboard allowing users to save favorite service listings and track relocation workflows, providing a tailored experience.	Registered Users
Review System	Users can rate and review service listings, maintaining service quality accountability and providing social proof to help future users make informed decisions.	Registered Users
Admin Control Panel	Secure admin interface for adding new cities, managing service listings, and overseeing platform-wide data. Protects data integrity across the platform.	Admin Only

4. Project Structure

The project follows a monorepo layout under relocatehub/version2/, with a clean separation between the backend API server and the frontend React application.

4.1 Backend Structure

`backend/`	Root of the Express.js API server application
`backend/server.js`	Main entry point — initialises Express, connects to MongoDB, mounts all route modules, and starts the HTTP server on the configured port
`backend/middleware/`	Authentication middleware — contains JWT validation logic used to protect secured routes, verifying tokens from incoming cookies

backend/models/	Mongoose schema definitions for all core data entities: User, City, Listing (service listing), and Review
backend/routes/	Express route modules organised by resource: auth (login/register), cities, listings, and reviews — each maps HTTP verbs to controller logic
backend/.env	Environment configuration file (not committed to VCS) — stores PORT, MONGO_URI, JWT_SECRET, and NODE_ENV

4.2 Frontend Structure

frontend/src/	Root of the React application source code
frontend/src/main.jsx	Application entry point — mounts the React root and wraps the app in the AuthContext provider
frontend/src/App.jsx	Main application router — defines all client-side routes using React Router and maps them to page components
frontend/src/components/	Shared, reusable UI components used across multiple pages (e.g., navigation bar, cards, search inputs, buttons)
frontend/src/context/	React Context module — AuthContext provides authenticated user state and helper functions globally to all child components
frontend/src/pages/	Route-level view components: Home, Dashboard, AdminPanel, DetailPage, and others — each corresponds to a URL route

5. Data Models

RelocateHub defines four primary Mongoose models that represent the core entities of the platform. These schemas enforce data integrity and relationships within MongoDB.

5.1 User Model

Stores account credentials and saved listings for registered users.

Field	Type	Description
username	String	Unique display name for the user account
email	String	Unique email address used for login identification
password	String	Bcrypt-hashed password (plaintext never stored)
role	String (Enum)	User role — either 'user' (default) or 'admin'
savedListings	Array (Ref)	References to saved service listing documents

5.2 City Model

Represents a destination city with all guide-level information.

- City name, description, region/country identifiers
- Cost of living data (overall index, housing, food, transport)
- Key neighborhoods and their characteristics
- Local essentials and quality-of-life metadata
- Population and climate summary data

5.3 Listing (Service) Model

Represents an individual service entry such as a housing property, school, or healthcare provider.

- Name, category (housing / school / healthcare), address, description
- Rating (aggregate, auto-calculated from reviews)
- Popularity score for real-time trending sort
- City reference (foreign key to City model)
- Contact details, website, and operating hours

5.4 Review Model

Captures user-submitted ratings and feedback for individual service listings.

- Author reference (User model foreign key)
- Listing reference (Listing model foreign key)
- Numeric rating (1–5 scale)
- Written review text body
- Timestamp of submission

6. API Routes

The backend exposes a RESTful API organised into four route groups. Protected routes require a valid JWT cookie (enforced by the auth middleware).

6.1 Authentication Routes — /api/auth

Method	Endpoint	Auth Required	Description
POST	/api/auth/register	No	Register a new user account
POST	/api/auth/login	No	Authenticate and set JWT cookie

POST	/api/auth/logout	Yes	Invalidate session and clear cookie
GET	/api/auth/me	Yes	Retrieve the authenticated user profile

6.2 City Routes — /api/cities

- GET /api/cities — Retrieve all city records
- GET /api/cities/:id — Get a single city by ID with full guide data
- POST /api/cities — Add a new city (Admin only)
- PUT /api/cities/:id — Update an existing city (Admin only)
- DELETE /api/cities/:id — Remove a city (Admin only)

6.3 Listing Routes — /api/listings

- GET /api/listings — Retrieve all listings, supports query params for category, city, rating filter, and sort
- GET /api/listings/:id — Get a single listing with full details and reviews
- POST /api/listings — Create a new listing (Admin only)
- PUT /api/listings/:id — Update a listing (Admin only)
- DELETE /api/listings/:id — Remove a listing (Admin only)
- POST /api/listings/:id/save — Save/unsave a listing to the current user's dashboard

6.4 Review Routes — /api/reviews

- GET /api/reviews?listing=:id — Get all reviews for a specific listing
- POST /api/reviews — Submit a new review (Authenticated users only)
- DELETE /api/reviews/:id — Delete a review (Author or Admin only)

7. Authentication & Security

7.1 JWT Cookie Authentication

RelocateHub implements a stateless authentication model using JSON Web Tokens (JWT). Upon successful login, the server signs a JWT containing the user ID and role, then sets it as an HTTP-only, Secure cookie on the response. This approach provides several advantages:

- XSS Protection — HTTP-only cookies are inaccessible to JavaScript, preventing token theft via cross-site scripting
- Stateless — The server does not maintain session state; every request is self-contained and independently verifiable
- Role-based Access — The JWT payload encodes the user role, enabling middleware to gate admin-only routes

- Persistence — Cookie-based tokens persist across browser sessions (until expiry), providing a smooth UX

7.2 Password Security

All user passwords are processed through bcrypt before storage. Bcrypt is an adaptive hashing function that:

- Incorporates a salt to prevent rainbow table attacks
- Uses a configurable work factor (cost), making brute-force computationally expensive
- Ensures plaintext passwords are never persisted to the database at any point

7.3 Auth Middleware

A dedicated middleware module intercepts requests to protected routes. It extracts the JWT from the cookie, verifies the signature using the JWT_SECRET, and attaches the decoded user payload to the request object before passing control to the route handler. Invalid or missing tokens result in a 401 Unauthorized response.

8. Local Setup & Installation

8.1 Prerequisites

Prerequisite	Details
Node.js	Version 16 or higher (LTS recommended). Download from nodejs.org.
npm	Bundled with Node.js. Used to install frontend and backend dependencies.
MongoDB	Running locally (default: mongodb://localhost:27017) or a MongoDB Atlas cloud connection string.
Git	Required to clone the repository from GitHub.

8.2 Installation Steps

Step 1: Clone the Repository

```
git clone https://github.com/yourusername/relocatehub.git
cd relocatehub/version2
```

Step 2: Backend Configuration

1. Navigate to the backend directory: cd backend
2. Install dependencies: npm install

3. Create a .env file with the following variables:

`PORT`	5000 (or any available port)
`MONGO_URI`	Your MongoDB connection string (local or Atlas)
`JWT_SECRET`	A long, random secret string for signing tokens
`NODE_ENV`	development

4. Start the backend server: npm start (runs on http://localhost:5000)

Step 3: Frontend Configuration

5. Navigate to the frontend directory: cd ../frontend
6. Install dependencies: npm install
7. Start the Vite dev server: npm run dev (runs on http://localhost:5173)

9. Potential Improvements & Roadmap

As RelocateHub matures, the following enhancements could significantly improve the platform's capability and user experience:

9.1 Technical Enhancements

- Rate Limiting & Helmet.js — Add express-rate-limit and helmet middleware for production-grade API hardening
- Email Verification — Implement account email verification flow via Nodemailer or a transactional email API (SendGrid, Resend)
- Image Uploads — Integrate Cloudinary or AWS S3 for listing and city image management
- Pagination — Add cursor or offset-based pagination for large listing datasets to improve API performance
- Input Validation — Introduce express-validator or Zod for server-side request body validation
- Testing — Add unit tests (Jest) for backend routes and integration tests for critical user flows

9.2 Feature Enhancements

- Interactive Maps — Embed Mapbox or Google Maps for geographical service discovery and city exploration
- Cost of Living Calculator — Dynamic tool comparing living costs between origin and destination cities

- Relocation Checklist — Guided step-by-step workflow for users to track their move progress
- Notifications — Real-time or email alerts for new listings matching a user's saved search criteria
- Social Sharing — Allow users to share city guides or service listings with prospective relocators
- Mobile App — React Native app to extend the platform's reach to iOS and Android users

10. License

RelocateHub is released under the MIT License, one of the most permissive and widely used open-source licenses. Under the MIT License:

- Anyone may use, copy, modify, merge, publish, distribute, sublicense, or sell copies of the software
- The only requirement is that the original copyright notice and this license text are included in all substantial copies
- The software is provided 'as is', without warranty of any kind, express or implied

This makes RelocateHub freely available for both personal and commercial use, while maintaining the author's attribution rights.

— End of Report —